SMART IRRIGATION SYSTEM USING ESP-32 & TESTING THE CROP DISEASE USING ML.

A Project Report Submitted in Partial Fulfillment of
the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY(B.TECH)

BY

G.MANAS SANHITH(2300049013).

T.NAGA SUDHARSHAN(2300049014).

K.VIGNESH(2300049015).

A.YUVA KIRAN(2300049016).

Under esteemed guidance of

PROF A.V PRABHU

K L University

u/s 3 of UGC Act, 1956

Koneru Lakshmaiah Education Foundation

Green Fields, Vaddeswaram, Guntur,

AndhraPradesh-522302

2026-2027

KLEF Deemed to be university,

Green fields vaddeswaram,

Guntur district,A.P.,INDIA..

BONAFIDE CERTIFICATE

Certified that this project report titled "**SMART IRRIGATION SYSTEM USING ESP-32 & TESTING CROP DISEASE TESTING USING ML.**" is the bonafide work of **G.MANAS SANHITH(RegisterNo:2300049013),T.NAGASUDHARSHAN(2300049014) ,K.VIGNESH(2300049015),A.YUVA KIRAN(2300049016)**who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Abstract

Agriculture plays a vital role in the economic growth of developing countries, where efficient utilization of water resources and early detection of crop diseases are major challenges. Traditional irrigation methods often lead to excessive water consumption and lack of timely monitoring, while crop diseases are usually identified manually, resulting in delayed treatment and reduced yield. To address these issues, this paper proposes a **Smart Irrigation System using ESP32 integrated with Machine Learning-based Crop Disease Detection**.

The proposed system continuously monitors soil moisture, temperature, and humidity parameters using appropriate sensors. Based on real-time sensor data, the ESP32 microcontroller automatically controls the irrigation process, ensuring optimal water supply to crops and preventing water wastage. The system is enabled with Internet of Things (IoT) technology, allowing farmers to remotely monitor field conditions and irrigation status through a mobile or web application.

In addition to smart irrigation, the system incorporates a Machine Learning (ML) model for crop disease detection. Images of crop leaves are captured using a camera module and processed using a trained ML model to identify common plant diseases. The detected disease name and confidence level are communicated to the user through the IoT platform, enabling early diagnosis and timely preventive measures. This combined approach enhances crop health monitoring and reduces dependency on manual inspection.

Experimental results demonstrate that the proposed system efficiently optimizes water usage while accurately detecting crop diseases under real-world conditions. The integration of IoT and Machine Learning provides a cost-effective, scalable, and intelligent solution suitable for modern precision agriculture.

HARDWARE AND SOFTWARE COMPONENTS.

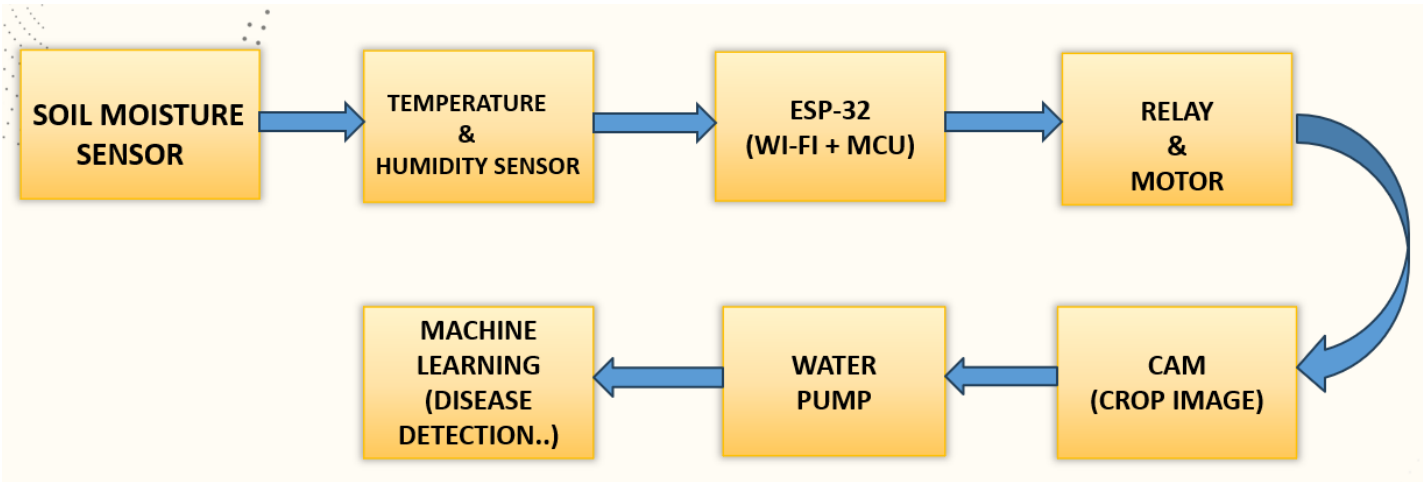
HARDWARE COMPONENTS:

- **ESP-32 Development Board**
- **Soil Moisture Sensor**
- **Temperature & Humidity Sensor (DHT11/DHT22)**
- **Camera**
- **Relay Module**
- **Water Pump / DC Motor**
- **Power Supply (5V / 12V)**
- **Connecting Wires & Breadboard**

SOFTWARE COMPONENTS:

- **Arduino IDE**
- **Google collab**

BLOCK DIAGRAM



1. Soil Moisture Sensor

The soil moisture sensor is used to measure the water content present in the soil. It continuously monitors the moisture level around the crop roots and generates an electrical signal based on the soil condition. When the soil is dry, the sensor produces a low moisture value, and when sufficient water is present, it produces a higher value. This sensor helps in determining the exact water requirement of the crops and prevents both under-irrigation and over-irrigation.

---

2. Temperature and Humidity Sensor

The temperature and humidity sensor monitors the surrounding environmental conditions of the agricultural field. It measures the ambient temperature and relative humidity, which are important factors affecting crop growth and disease occurrence. These parameters help the system in understanding weather conditions and adjusting irrigation accordingly. High temperature and low humidity conditions indicate increased water demand for crops.

---

3. ESP32 (Wi-Fi Enabled Microcontroller)

The ESP32 acts as the central processing unit of the system. It collects real-time data from the soil moisture sensor and the temperature and humidity sensor. Based on predefined threshold values, the ESP32 decides whether irrigation is required or not. It also controls the relay module to operate the water pump. The built-in Wi-Fi feature enables the ESP32 to send sensor data and system status to a cloud platform or mobile application for remote monitoring.

---

4. Relay and Motor Control Unit

The relay module is used to safely control high-power devices such as the water pump using low-power signals from the ESP32. Since the ESP32 operates at low voltage levels, it cannot directly drive the motor. The relay provides electrical isolation between the control circuit and the power circuit, ensuring safe and reliable operation of the irrigation system.

---

5. Water Pump

The water pump is responsible for supplying water to the crops during irrigation. When the ESP32 detects that the soil moisture level is below the required threshold, it activates the relay, which turns ON the water pump. The pump supplies water through pipes or drip irrigation until the soil reaches the desired moisture level, after which the pump is automatically turned OFF.

---

6. Camera Module (Crop Image Capture)

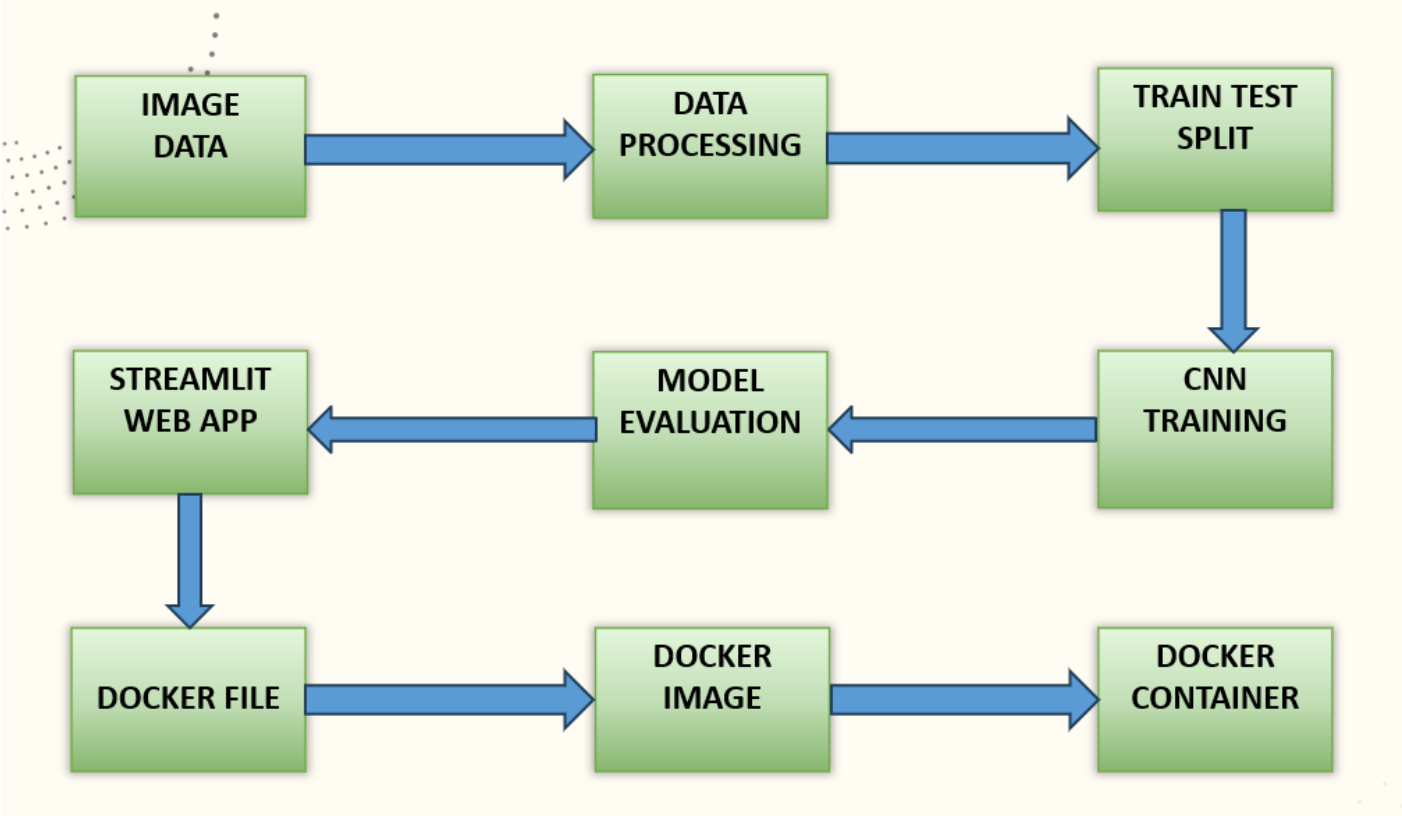
The camera module is used to capture images of crop leaves for disease analysis. Periodic images are taken and provided to the machine learning system for further processing. Visual features such as color changes, spots, and texture variations in leaves are important indicators of plant health and disease conditions.

---

7. Machine Learning Based Crop Disease Detection

The machine learning module analyzes the captured crop images to detect and classify plant diseases. A trained ML model processes the images and identifies whether the crop is healthy or affected by a specific disease. The detected disease information is sent to the user through an IoT platform, enabling early diagnosis and timely treatment. This intelligent approach reduces crop loss and improves agricultural productivity.

WORK FLOW



1. Image Data

Image data forms the foundation of the crop disease detection system. It consists of leaf images collected from agricultural fields or public datasets. These images include both healthy and diseased crop leaves. The quality and diversity of the image dataset play a crucial role in improving the accuracy of the machine learning model. The images serve as input for training and testing the disease classification system.

2**. Data Processing**

Data processing involves preparing raw images for machine learning analysis. This step includes image resizing, normalization, noise removal, and data augmentation techniques such as rotation, flipping, and scaling. These operations improve model robustness and help reduce overfitting. Data processing ensures that all images are in a consistent format suitable for CNN-based training.

---

3. Train–Test Split

The processed dataset is divided into training and testing subsets. The training set is used to train the convolutional neural network, while the testing set is used to evaluate model performance on unseen data. This separation helps in assessing the generalization capability of the model and prevents biased performance evaluation.

---

4. CNN Training

Convolutional Neural Network (CNN) training is the core stage of the disease detection system. The CNN automatically extracts important features such as color variations, leaf texture, and disease patterns from the images. During training, the model learns to associate these features with corresponding disease classes by minimizing loss and improving accuracy through multiple training epochs.

---

5. Model Evaluation

Model evaluation is performed to assess the effectiveness of the trained CNN. Performance metrics such as accuracy, precision, recall, and F1-score are calculated using the test dataset. This step ensures that the model is reliable and capable of correctly identifying crop diseases in real-world conditions.

---

6. Streamlit Web Application

The Streamlit web application provides an interactive user interface for disease detection. Users can upload crop leaf images through the web interface, and the trained CNN model predicts the disease class in real time. This application makes the system user-friendly and accessible to farmers and agricultural experts without requiring technical knowledge.

---

7. Docker File

The Dockerfile defines the software environment required to run the Streamlit application and the trained machine learning model. It specifies dependencies, libraries, and system configurations. Using a Dockerfile ensures consistency across different deployment platforms and eliminates environment compatibility issues.

---

8. Docker Image

The Docker image is a packaged version of the application created using the Dockerfile. It contains the trained CNN model, Streamlit web application, and all necessary dependencies. This image enables easy distribution and deployment of the system on any Docker-supported platform.

9. Docker Container

The Docker container is a running instance of the Docker image. It executes the crop disease detection application in an isolated and controlled environment. Containers ensure scalability, portability, and efficient resource utilization, making the system suitable for real-world deployment.

The proposed workflow describes a systematic approach for detecting crop diseases using image-based machine learning techniques. The system integrates data acquisition, deep learning-based image classification, and web-based deployment to provide an efficient and scalable solution for agricultural disease monitoring.

The workflow begins with the collection of image data consisting of healthy and diseased crop leaf images. These images capture visible symptoms such as discoloration, spots, and texture variations, which are critical indicators of plant diseases. High-quality and diverse image datasets improve the learning capability of the model and enhance detection accuracy.

In the data processing stage, the collected images are preprocessed to make them suitable for deep learning analysis. Preprocessing includes resizing images to a fixed resolution, normalizing pixel values, and applying data augmentation techniques such as rotation and flipping. These steps help reduce noise, handle variations in image conditions, and prevent overfitting during model training.

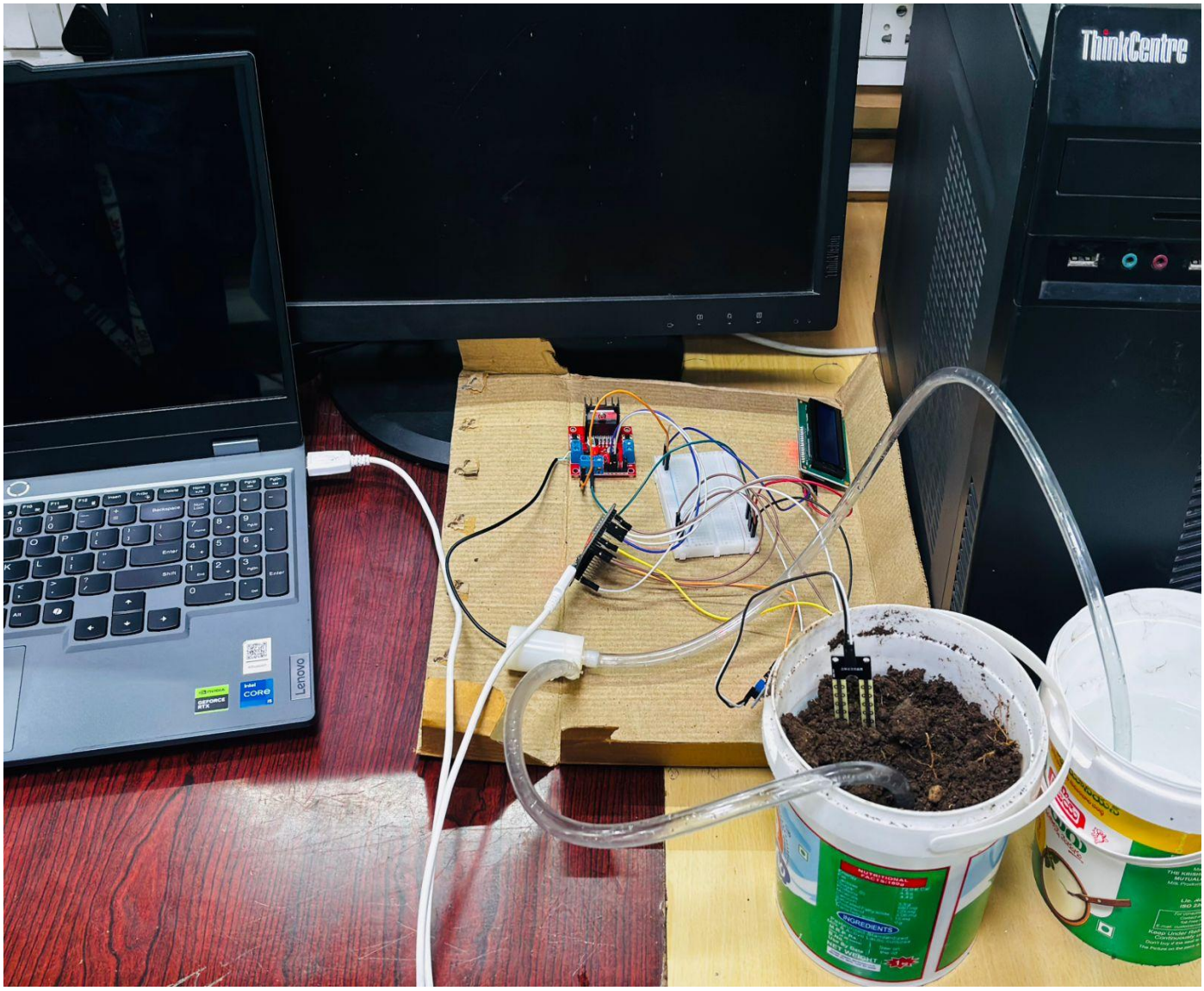
CODE

```
import os
import json
from PIL import Image

import numpy as np
import tensorflow as tf
import streamlit as st



working_dir = os.path.dirname(os.path.abspath(__file__))
model_path = f"{working_dir}/trained_model/plant_disease_prediction_model.keras"
# Load the pre-trained model
model = tf.keras.models.load_model(model_path, compile=False)

   # loading the class names
class_indices = json.load(open(f"{working_dir}/class_indices.json"))



   # Function to Load and Preprocess the Image using Pillow
def load_and_preprocess_image(image_path, target_size=(224, 224)):
       # Load the image
       img = Image.open(image_path)
       # Resize the image
       img = img.resize(target_size)
       # Convert the image to a numpy array
```

```
    img_array = np.array(img)
    # Add batch dimension
    img_array = np.expand_dims(img_array, axis=0)
    # Scale the image values to [0, 1]
    img_array = img_array.astype('float32') / 255.
    return img_array


# Function to Predict the Class of an Image
def predict_image_class(model, image_path, class_indices):
    preprocessed_img = load_and_preprocess_image(image_path)
    predictions = model.predict(preprocessed_img)
    predicted_class_index = np.argmax(predictions, axis=1)[0]
    predicted_class_name = class_indices[str(predicted_class_index)]
    return predicted_class_name


# Streamlit App
st.title('Plant Disease Classifier')

uploaded_image = st.file_uploader("Upload an image...", type=["jpg", "jpeg", "png"])

if uploaded_image is not None:
    image = Image.open(uploaded_image)
    col1, col2 = st.columns(2)

    with col1:
        resized_img = image.resize((150, 150))
        st.image(resized_img)
```

```
with col2:
    if st.button('Classify'):
    # Preprocess the uploaded image and predict the class
        prediction = predict_image_class(model, uploaded_image, class_indices)
        st.success(f'Prediction: {str(prediction)}')
```

OUTPUT



TEAM MEMBER DETAILS

G.MANAS SANHITH(2300049013)

T.NAGA SUDHARSHAN(2300049014)

K.VIGNESH(2300049015)

A.YUVA KIRAN(2300049016)